LAPORAN PRAKTIKUM

NATURAL LANGUAGE PROCESSOR

Annisa Almaghvirah

240306037

https://github.com/ANNISAALMAGHVIRAH

Program Studi Teknologi Informasi

Fakultas Dakwah dan Ilmu Komunikasi

Universitas Islam Negeri Mataram

2026

Modul	Modul 2 - Representasi Teks dan Neural NLP
Topik	Representasi Teks: BoW dan TF-IDF
Repository	NATURAL-LANGUAGE-PROCESSING/MODUL 2 at main · ANNISAALMAGHVIRAH/NATURAL-LANGUAGE-PROCESSING
Tanggal	20 Oktober 2026

A. Tujuan

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Memahami mengapa teks perlu diubah ke bentuk numerik
2. Membuat representasi Bag of Words (BoW)
3. Menghitung Term Frequency (TF) dan Inverse Document Frequency (IDF)
4. Membuat matriks TF-IDF menggunakan scikit-learn
5. Menghitung cosine similarity antar dokumen
6. Memahami keterbatasan representasi berbasis frekuensi

B. Requiretment

1. Sistem Operasi yang digunakan : Windows 11 Home Single Language 64-bit (10.0, Build 26100)
2. Browser : Google Chrome Version 130.0.6723.117 (Official Build) (64-bit)
3. Tools yang digunakan : Visual Studio Code, Python , Jupyter Notebook, Virtual Environment (.venv), pandas, numpy, matplotlib, seaborn, nltk

C. Dasar Teori

1. Mengapa Teks Harus Diubah ke Numerik?

   Model Machine Learning dan Neural Network tidak dapat memproses teks mentah secara langsung. Teks harus diubah menjadi representasi numerik.

```
"Saya belajar NLP"  →  [0, 1, 0, 1, 0, 1]  (BoW)
                    →  [0.0, 0.45, 0.0, 0.63, 0.0, 0.45]  (TF-IDF)
```

2. Vocabulary

   Vocabulary adalah kumpulan semua kata unik dalam corpus.

   Contoh corpus:

```
Dokumen 1: "Saya belajar NLP"
Dokumen 2: "Saya belajar Python"
Dokumen 3: "NLP menggunakan Python"
```

   Vocabulary:

```
["Saya", "belajar", "NLP", "Python", "menggunakan"]
```

3. **Bag of Words (BoW)**

   BoW merepresentasikan setiap dokumen sebagai vektor frekuensi kata dari vocabulary.

   Contoh:

Dokumen	Saya	belajar	NLP	Python	menggunakan
D1	1	1	1	0	0
D2	1	1	0	1	0
D3	0	0	1	1	1

   Keterbatasan BoW:

   - Tidak mempertimbangkan urutan kata (order)
   - Tidak membedakan kata penting dan kata umum
   - Dimensi sangat besar (seukuran vocabulary)
   - Matriks sangat sparse (banyak nilai nol)

4. Term Frequency (TF)

   TF mengukur seberapa sering sebuah kata muncul dalam satu dokumen:

$$TF(t,d) = \frac{\text{Jumlah kemunculan } t \text{ dalam } d}{\text{Total kata dalam } d}$$

5. Inverse Document Frequency (IDF)

   IDF mengukur seberapa langka sebuah kata di seluruh corpus:

$$IDF(t) = \log\left(\frac{N}{df(t)}\right)$$

   dengan:

   - N = jumlah total dokumen
   - df(t) = jumlah dokumen yang mengandung kata t

   Kata yang muncul di banyak dokumen (seperti stopword) akan mendapat IDF rendah. Kata yang langka mendapat IDF tinggi.

6. TF-IDF

   TF-IDF adalah perkalian TF dan IDF:

$$TFIDF(t, d) = TF(t, d) \times IDF(t)$$

   TF-IDF memberikan bobot tinggi pada kata yang sering muncul dalam dokumen tertentu, tetapi jarang di seluruh corpus — kata tersebut dianggap kata kunci dokumen.

7. Cosine Similarity

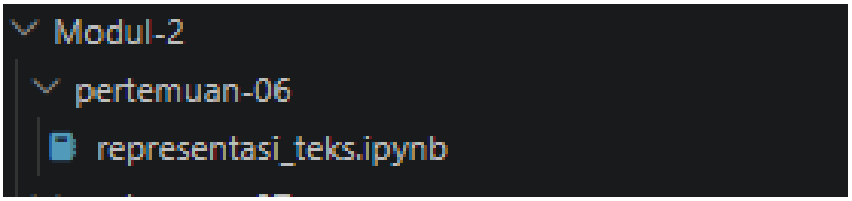
   Cosine similarity mengukur kemiripan antara dua vektor berdasarkan sudut di antara keduanya:

$$\cos(\theta) = \frac{A \cdot B}{|A| \cdot |B|}$$

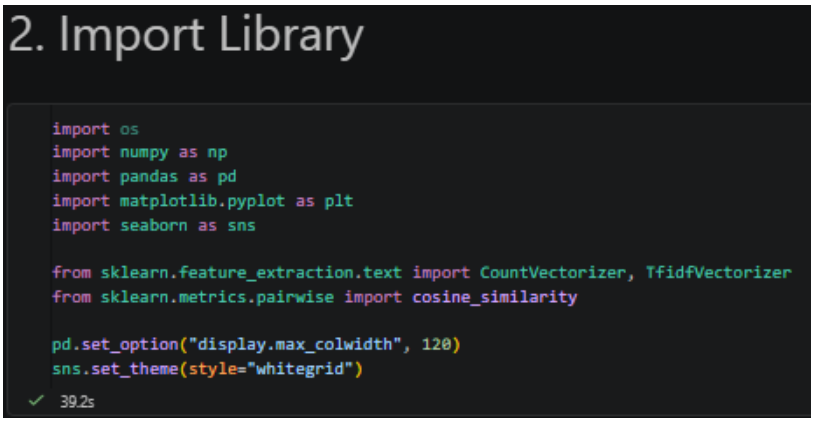
   Nilai:

   - = dokumen identik
   - = dokumen sama sekali berbeda
   - -1.0 = dokumen berlawanan arah (jarang terjadi pada representasi positif)

D. Implementasi



Struktur folder terbagi secara hierarkis mulai dari direktori utama Modul-2, yang di dalamnya berisi sub-folder pertemuan-06. Di dalam folder pertemuan tersebut, terdapat satu berkas notebook Jupyter bernama representasi_teks.ipynb.



Perintah ini diawali dengan mengimpor pustaka dasar untuk penanganan sistem operasi (`os`), komputasi numerik (numpy), serta manipulasi data (`pandas`). Pustaka visualisasi data juga diimpor menggunakan perintah `import matplotlib.pyplot as plt` dan `import seaborn as sns`.

Perintah ini kemudian dilanjutkan dengan mengimpor modul dari `scikit-learn` untuk kebutuhan ekstraksi fitur teks dan perhitungan kemiripan dokumen. Pengimporan mencakup `CountVectorizer` dan `TfidfVectorizer` melalui `from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer` serta `cosine_similarity` via `from sklearn.metrics.pairwise import cosine_similarity`.

Perintah ini diakhiri dengan mengonfigurasi pengaturan tampilan dataframe menggunakan perintah `pd.set_option("display.max_colwidth", 120)` untuk membatasi lebar maksimum kolom teks menjadi 120 karakter, serta menerapkan tema visualisasi grafik berlatar belakang kisi-kisi putih melalui `sns.set_theme(style="whitegrid")`.

3. Load Dataset

```
import pandas as pd

data = pd.read_csv("D:/NLP/Modul-2/dataset_preprocessed.csv")
text_column = "processed_text"

data = data.dropna(subset=[text_column])

# Gunakan sample untuk eksperimen awal
sample = data[text_column].head(1000).astype(str).tolist()
print(f"Jumlah dokumen sample: {len(sample)}")

print("Ukuran dataset:", data.shape)
print("Kolom:", data.columns.tolist())
```

✓ 0.6s

```
Jumlah dokumen sample: 1000
Ukuran dataset: (119676, 2)
Kolom: ['content', 'processed_text']
```

```
# Pilih kolom teks hasil preprocessing
data = data.copy()

preferred_text_columns = [
    "processed_text", "stemming", "cleaning", "content", "text"
]
text_column = next(
    (column for column in preferred_text_columns if column in data.columns),
    None
)

if text_column is None:
    object_columns = data.select_dtypes(
        include=["object", "string"]
    ).columns.tolist()
print("Kolom teks terpilih:", text_column)

data[text_column] = data[text_column].fillna("").astype(str).str.strip()
data_text = data[data[text_column].ne("")].copy()

# Hapus duplikat teks untuk mengurangi dokumen yang sama persis berulang.
# Jika ingin mempertahankan semua ulasan, hapus baris drop_duplicates ini.
data_text = data_text.drop_duplicates(subset=[text_column]).reset_index(drop=True)

print("Jumlah dokumen teks yang valid dan unik:", len(data_text))
sample_size = min(1000, len(data_text))
sample = data_text[text_column].head(sample_size).tolist()
print(f"Jumlah dokumen sample untuk eksperimen: {len(sample)}")
```

✓ 0.4s

```
Kolom teks terpilih: processed_text
Jumlah dokumen teks yang valid dan unik: 65556
Jumlah dokumen sample untuk eksperimen: 1000
```

Perintah ini diawali dengan mengimpor pustaka `pandas` sebagai `pd` untuk memanipulasi data, lalu membaca berkas CSV dari direktori yang ditentukan ke dalam DataFrame data. Nama kolom teks target diset ke `"processed_text"`, lalu baris yang memiliki nilai kosong (*NaN*) pada kolom tersebut dibuang melalui `dropna()`. Untuk eksperimen awal, diambil 1000 data pertama dari kolom teks target yang diubah ke bentuk *list* string, diikuti dengan pencetakan informasi jumlah dokumen sampel, ukuran/dimensi DataFrame (119.676 baris dan 2 kolom), serta daftar kolom yang tersedia.

Perintah ini kemudian dilanjutkan dengan menyalin DataFrame awal ke variabel `data` baru untuk proses seleksi kolom teks secara dinamis. Sebuah daftar prioritas nama kolom (`preferred_text_columns`) disiapkan, lalu fungsi `next()` mengekstrak kolom pertama yang ada pada dataset. Jika tidak ada nama kolom

yang cocok, blok pengondisian `if text_column is None` akan mendeteksi kolom berjenis *object* atau *string* secara otomatis. Setelah kolom `"processed_text"` dipastikan terpilih, nilai *missing* diisi string kosong, tipe data dipastikan string, dan spasi berlebih dibersihkan via `fillna("").astype(str).str.strip()`, lalu data yang tidak kosong disaring ke dalam DataFrame `data_text`.

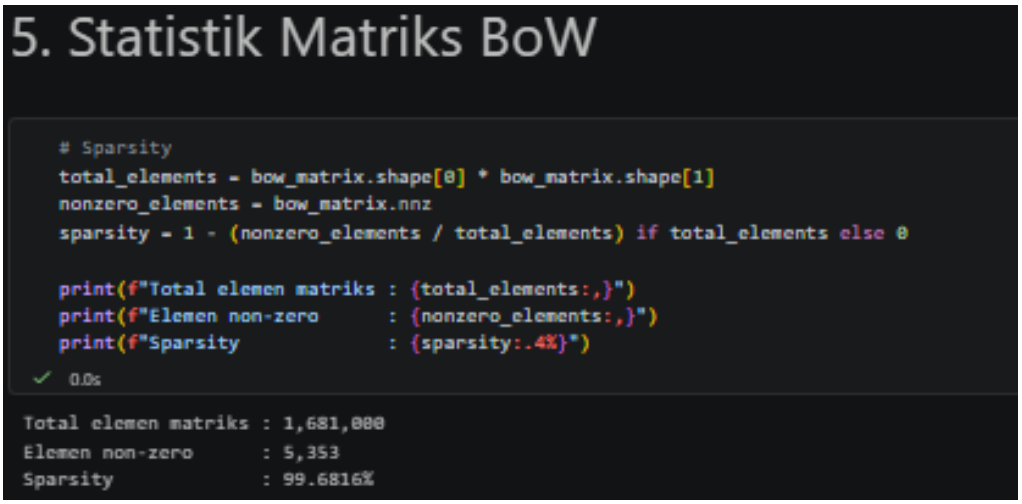
Perintah ini diakhiri dengan mengeliminasi teks ulasan yang duplikat menggunakan fungsi `drop_duplicates()` dan memperbarui urutan indeksnya agar diperoleh seluruh dokumen yang unik. Hasil akhir pembersihan mencetak kolom terpilih, jumlah total dokumen unik dan valid (tersisa 65.556 dokumen), serta menentukan ukuran sampel eksperimen dengan fungsi `min(1000, len(data_text))`. Sebanyak 1.000 sampel dokumen teks unik berhasil diekstrak dan siap digunakan untuk tahap ekstraksi fitur berikutnya.



Perintah ini diawali dengan menginisialisasi objek *vectorizer* menggunakan `vectorizer_bow = CountVectorizer(max_features=5000)` untuk membatasi kosa kata maksimum sebanyak 5.000 kata unik. Objek tersebut kemudian mempelajari kosa kata sekaligus mentransformasikan data teks sampel

menjadi matriks frekuensi kata melalui perintah `bow_matrix = vectorizer_bow.fit_transform(sample)`. Selanjutnya, daftar seluruh kata unik yang berhasil diekstrak diambil menggunakan `vocab_bow = vectorizer_bow.get_feature_names_out()`. Bagian sel ini diakhiri dengan mencetak dimensi matriks BoW (terdiri dari 1.000 dokumen dan 1.681 kata unik), jumlah total kosakata (1.681 kata), serta menampilkan 20 contoh kata pertama dalam alfabetis.

Perintah ini kemudian dilanjutkan dengan memeriksa bentuk representasi vektor untuk dokumen pertama. Baris pertama dari matriks sparse diekstrak dan diubah menjadi *array* 1D melalui `doc_vector = bow_matrix[0].toarray()[0]`, lalu indeks kata-kata yang muncul (frekuensi > 0) dideteksi menggunakan `nonzero_indices = np.where(doc_vector > 0)[0]`. Informasi teks asli dari dokumen pertama (yaitu kata `"mantap"`) dicetak ke layar lewat `print(sample[0])`. Sebagai penutup, perulangan `for idx in nonzero_indices:` mengeksekusi pencetakan nama kata beserta frekuensi kemunculannya dalam dokumen tersebut, yang menampilkan bahwa kata `"mantap"` muncul sebanyak 1 kali.



Perintah ini diawali dengan menghitung total elemen matriks melalui perkalian jumlah dokumen dan jumlah kosakata via `total_elements = bow_matrix.shape[0] * bow_matrix.shape[1]`, serta mengambil jumlah elemen bernilai tidak nol (*non-zero*) menggunakan properti `nonzero_elements = bow_matrix.nnz`. Selanjutnya, tingkat kekosongan matriks dihitung melalui

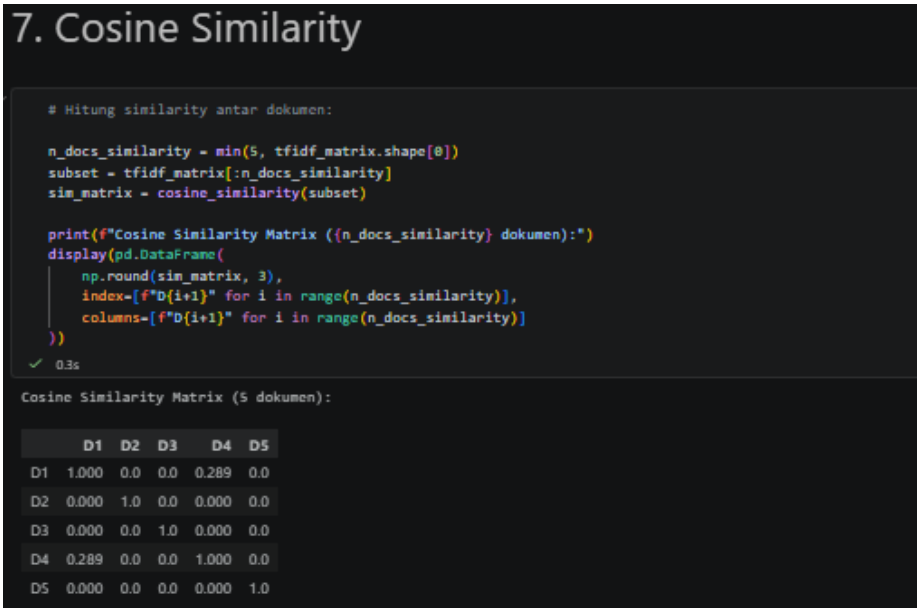
rumus `sparsity = 1 - (nonzero_elements / total_elements) if total_elements else 0` untuk mengetahui proporsi elemen bernilai nol dalam matriks *sparse* tersebut.

Perintah ini diakhiri dengan mencetak hasil kalkulasi statistik ke layar secara terstruktur. Cetakan mencakup total elemen matriks (1.681.000 elemen), jumlah elemen *non-zero* atau kata yang benar-benar muncul (5.353 elemen), serta persentase *sparsity* sebesar 99,6816% yang menunjukkan bahwa mayoritas matriks terisi oleh nilai nol.



Perintah ini diawali dengan menginisialisasi objek pembobot teks `vectorizer_tfidf = TfidfVectorizer(max_features=5000)` untuk membatasi kosa kata maksimum sebanyak 5.000 kata. Objek tersebut kemudian menghitung pembobotan TF-IDF sekaligus mentransformasikan data teks sampel menjadi matriks pembobotan melalui `tfidf_matrix = vectorizer_tfidf.fit_transform(sample)`. Selanjutnya, daftar nama seluruh fitur kata yang berhasil diekstrak diambil `menggunakan feature_names = vectorizer_tfidf.get_feature_names_out()`. Sel ini diakhiri dengan mencetak bentuk dimensi matriks TF-IDF (1.000 dokumen dan 1.681 fitur) serta jumlah total fitur kata yang terbentuk.

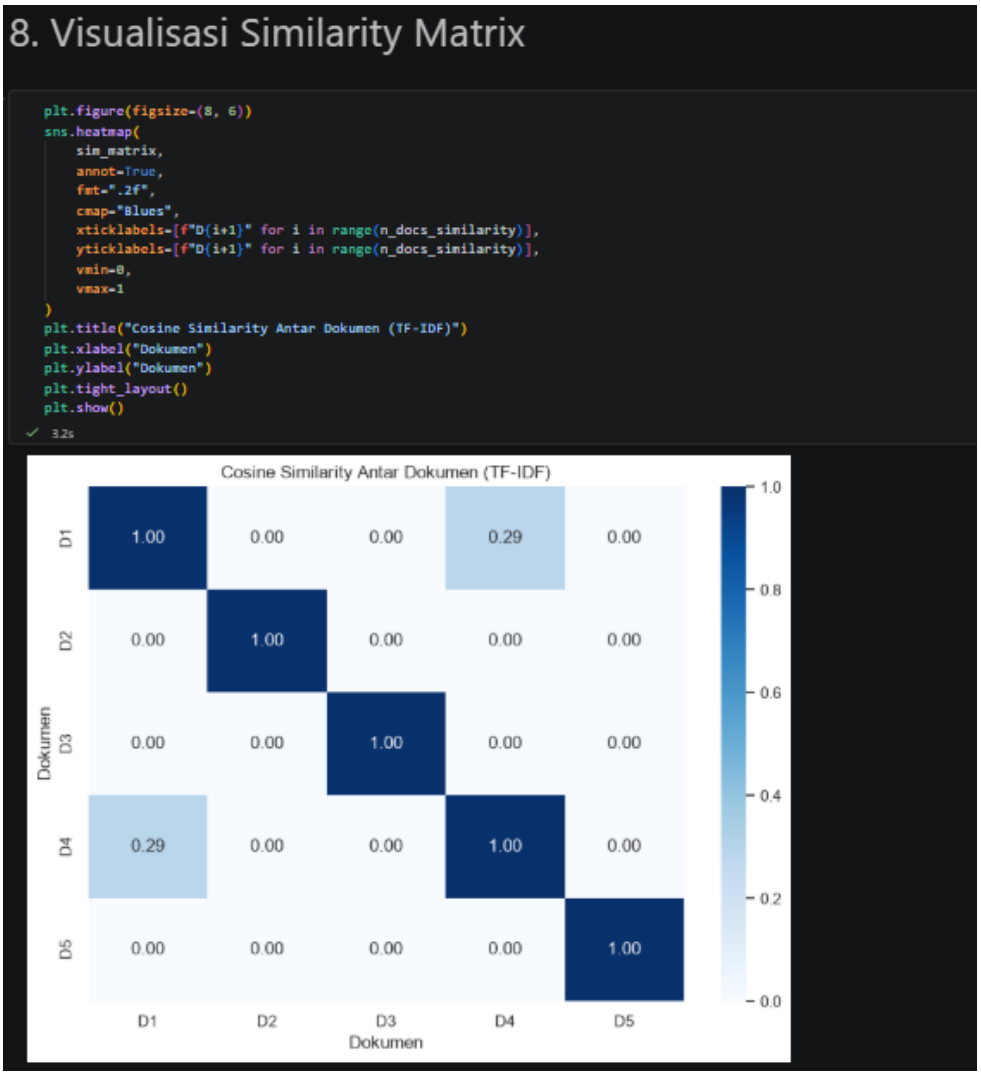
Perintah ini kemudian dilanjutkan dengan mendefinisikan fungsi `top_tfidf_words()` untuk mengekstrak kata-kata dengan skor TF-IDF tertinggi pada dokumen tertentu. Di dalam fungsi, vektor dokumen diubah menjadi *array* 1D via `row = tfidf_mat[doc_index].toarray().ravel()`, lalu indeksnya diurutkan secara menurun berdasarkan skor terbesar melalui `indices = row.argsort()[::-1]`. Fungsi lalu mengembalikan pasangan nama kata dan nilai skor TF-IDF yang lebih besar dari nol sebanyak *N* teratas. Kode diakhiri dengan mencetak isi teks dokumen pertama serta menjalankan perulangan `for word, score in top_tfidf_words(...)` untuk menampilkan daftar kata berbobot tertinggi, di mana kata `"mantap"` pada dokumen pertama memperoleh skor TF-IDF maksimum sebesar 1.0000.



Perintah ini diawali dengan menentukan jumlah dokumen yang akan dihitung kemiripannya melalui `n_docs_similarity = min(5, tfidf_matrix.shape[0])`, yaitu mengambil nilai terkecil antara 5 dokumen atau total dokumen yang ada. Vektor dari 5 dokumen pertama kemudian diekstrak ke dalam variabel `subset = tfidf_matrix[:n_docs_similarity]`. Nilai kemiripan sudut antar-dokumen lalu dihitung menggunakan fungsi `cosine_similarity(subset)` dan hasilnya disimpan ke dalam variabel `sim_matrix`.

Perintah ini diakhiri dengan mengonversi matriks hasil perhitungan menjadi bentuk DataFrame pandas agar tampilannya lebih rapi dan mudah dibaca. Setiap

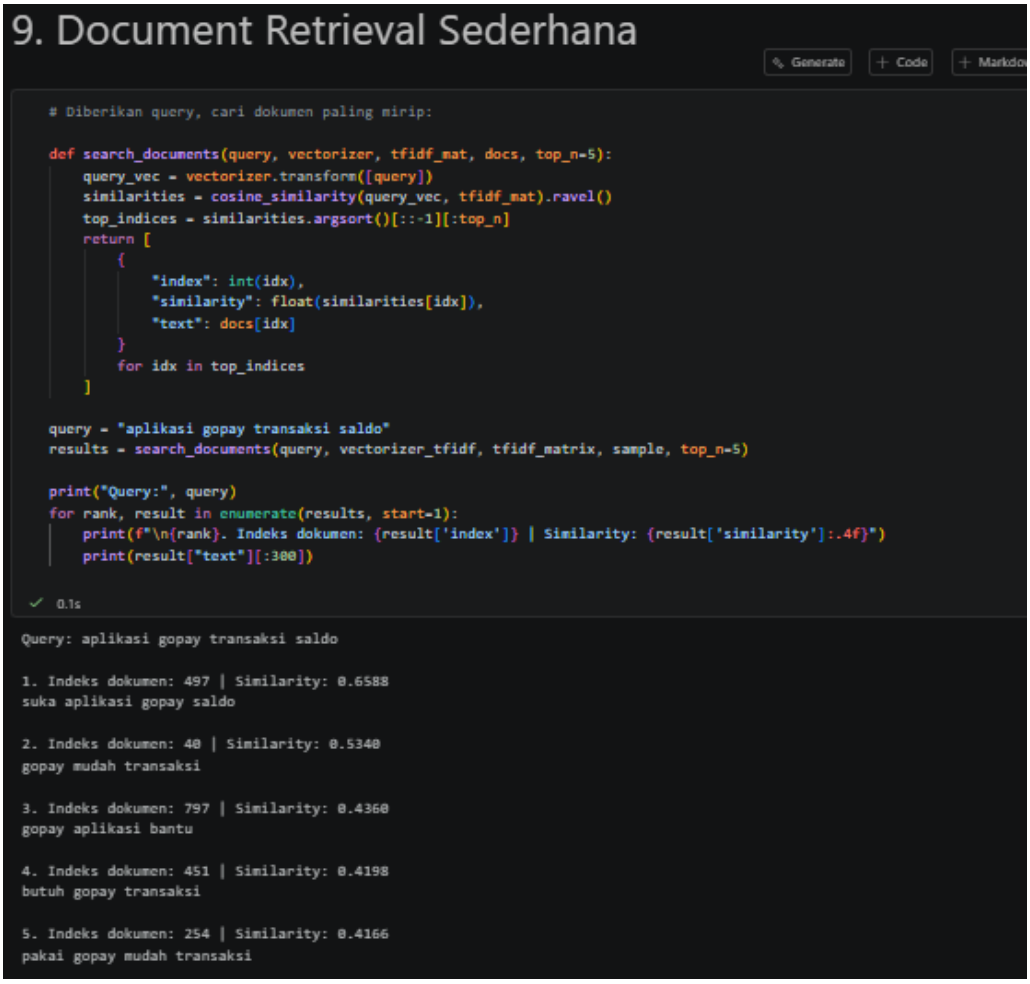
baris dan kolom diberi label nama dokumen secara otomatis (`D1` hingga `D5`) dengan tingkat pembulatan 3 angka di belakang koma via `np.round(sim_matrix, 3)`. Tampilan tabel matriks menunjukkan nilai kemiripan sempurna 1.0 pada diagonal utama (dokumen dengan dirinya sendiri), serta menunjukkan adanya kemiripan kata sebesar 0.289 antara dokumen pertama (`D1`) dan dokumen keempat (`D4`).



Perintah ini diawali dengan mengatur ukuran kanvas grafik menggunakan `plt.figure(figsize=(8, 6))` dan memvisualisasikan matriks kemiripan ke dalam bentuk *heatmap* menggunakan fungsi `sns.heatmap()`. Di dalam fungsi *heatmap*, variabel `sim_matrix` dimasukkan bersama argumen `annot=True` dan `fmt=".2f"` untuk menampilkan angka nilai kemiripan dengan format dua digit desimal pada setiap sel matriks. Gradasi warna diatur menggunakan skema biru

(`cmap="Blues"`), sedangkan rentang skala warna dibatasi dari nilai terkecil 0 (`vmin=0`) hingga nilai tertinggi 1 (`vmax=1`). Sumbu horizontal dan vertikal kemudian diberi label indeks dokumen `D1` hingga `D5` secara dinamis melalui parameter `xticklabels` dan `yticklabels`.

Perintah ini diakhiri dengan menambahkan atribut pelengkap visualisasi untuk memperjelas informasi grafik. Judul grafik ditentukan via `plt.title("Cosine Similarity Antar Dokumen (TF-IDF)")`, sementara nama sumbu horizontal dan vertikal diset menggunakan `plt.xlabel("Dokumen")` dan `plt.ylabel("Dokumen")`. Selanjutnya, tata letak grafik dirapikan secara otomatis menggunakan `plt.tight_layout()` agar label tidak terpotong, lalu tampilan *heatmap* diproyeksikan ke layar via perintah `plt.show()`. Visualisasi matriks ini memperjelas gradasi warna biru tua pada diagonal utama (kemiripan bernilai 1.00) serta warna biru muda pada sel `D1-D4` dan `D4-D1` yang mewakili nilai kemiripan sebesar 0.29.



Perintah ini diawali dengan merancang fungsi pencarian `search_documents()` untuk menemukan dokumen paling relevan berdasarkan input kata kunci (*query*). Di dalam fungsi, Teks *query* terlebih dahulu ditransformasikan ke bentuk vektor TF-IDF menggunakan perintah `query_vec = vectorizer.transform([query])`. Nilai kemiripan antara vektor *query* dengan seluruh dokumen dalam dataset kemudian dihitung via `similarities = cosine_similarity(query_vec, tfidf_mat).ravel()`. Indeks dokumen dengan skor kemiripan tertinggi lalu diurutkan secara menurun melalui `top_indices = similarities.argsort()[::-1][:top_n]`. Fungsi mengembalikan daftar kamus (*list of dictionary*) yang menyimpan informasi nomor indeks, skor *similarity*, dan isi teks dokumen teratas.

Perintah ini diakhiri dengan menguji fungsi menggunakan kata kunci pencarian `query = "aplikasi gopay transaksi saldo"` dan mengeksekusi fungsi pencarian via `results = search_documents(...)` untuk mengambil 5 dokumen paling mirip. Informasi *query* serta hasil pencarian selanjutnya dicetak ke layar secara terstruktur menggunakan perulangan `for rank, result in enumerate(results, start=1):`. Output menampilkan 5 dokumen peringkat teratas lengkap dengan indeksnya, di mana dokumen pada indeks ke-497 menduduki peringkat pertama dengan tingkat kemiripan paling tinggi sebesar 0.6588, disusul oleh dokumen-dokumen relevan lainnya.

10. Perbandingan BoW vs TF-IDF

```
print("-" * 50)
print("PERBANDINGAN BOW vs TF-IDF")
print("-" * 50)

# Ambil 1 dokumen
doc_idx = 0
bow_vec = vectorizer_bow.transform([sample[doc_idx]]).toarray()[0]
tfidf_vec = vectorizer_tfidf.transform([sample[doc_idx]]).toarray()[0]

bow_vocab = vectorizer_bow.get_feature_names_out()
tfidf_vocab = vectorizer_tfidf.get_feature_names_out()

print(f"\nDokumen: {sample[doc_idx][:100]}...")
print("\nTop 10 Kata — BoW (frekuensi):")
top_bow = sorted(
    [(bow_vocab[i], bow_vec[i]) for i in range(len(bow_vec)) if bow_vec[i] > 0],
    key=lambda x: x[1], reverse=True
)[:10]
for word, val in top_bow:
    print(f"  {word:<20} : {int(val)}")

print("\nTop 10 Kata — TF-IDF (bobot):")
for word, score in top_tfidf_words(doc_idx, tfidf_matrix, tfidf_vocab):
    print(f"  {word:<20} : {score:.4f}")
```

✓ 0.0s

```
--------------------------------------------------
PERBANDINGAN BOW vs TF-IDF
--------------------------------------------------

Dokumen: mantap...

Top 10 Kata — BoW (frekuensi):
  mantap               : 1

Top 10 Kata — TF-IDF (bobot):
  mantap               : 1.0000
```

Perintah ini diawali dengan mencetak header pembatas visual, lalu menentukan indeks dokumen yang akan diuji melalui `doc_idx = 0`. Teks dari dokumen tersebut kemudian diubah menjadi vektor numerik dalam bentuk *array* 1D, masing-masing menggunakan `vectorizer_bow.transform()` untuk metode Bag of Words dan `vectorizer_tfidf.transform()` untuk metode TF-IDF. Daftar kosakata unik dari kedua model ekstraksi fitur juga disiapkan ke dalam variabel `bow_vocab` dan `tfidf_vocab` menggunakan metode `.get_feature_names_out()`. Selanjutnya, potongan 100 karakter pertama dari teks dokumen acuan (`sample[doc_idx]`) dicetak ke layar.

Perintah ini diakhiri dengan mengekstrak dan membandingkan hasil fitur teratas dari kedua metode. Untuk Bag of Words, daftar kata berfrekuensi positif diurutkan secara menurun berdasarkan nilai kuncinya (`key=lambda x: x[1], reverse=True`) dan diambil 10 teratas. Hasilnya ditampilkan via perulangan `for word, val in top_bow:` yang memperlihatkan nilai frekuensi mutlak (misalnya kata `"mantap"` dengan frekuensi 1). Sementara itu, untuk TF-IDF, fungsi `top_tfidf_words()` dipanggil untuk menampilkan kata berbobot tertinggi. Hasil perbandingan ini memperlihatkan perbedaan di mana BoW menyajikan frekuensi kemunculan kata dalam bilangan bulat, sedangkan TF-IDF menyajikan pembobotan relatif dalam angka desimal (seperti kata `"mantap"` dengan skor 1.0000).

E. Tugas Analisis

```
# Persiapan – muat data dan buat representasi
import pandas as pd
import numpy as np
from sklearn.feature_extraction.text import CountVectorizer, TfidfTransformer
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.preprocessing import normalize
from IPython.display import display

data = pd.read_csv("D:/NLP/Modul-2/dataset_preprocessed.csv")
data = data.dropna(subset=["processed_text"]).copy()
data["processed_text"] = data["processed_text"].astype(str).str.strip()
data = data[data["processed_text"] != ""].reset_index(drop=True)

count_vectorizer = CountVectorizer()
bow_matrix = count_vectorizer.fit_transform(data["processed_text"])

tfidf_transformer = TfidfTransformer()
tfidf_matrix = tfidf_transformer.fit_transform(bow_matrix)

print("Jumlah dokumen:", len(data))
print("Ukuran kosakata:", len(count_vectorizer.get_feature_names_out()))
```

✓ 3.8s

```
Jumlah dokumen: 119613
Ukuran kosakata: 32654
```

Perintah ini diawali dengan mengimpor pustaka dasar untuk manipulasi data (`pandas` dan `numpy`), modul ekstraksi serta transformasi fitur teks (`CountVectorizer` dan `TfidfTransformer`), fungsi kalkulasi kemiripan (`cosine_similarity`), fungsi normalisasi vektor (`normalize`), serta modul `display` dari `IPython.display` untuk menampilkan data secara interaktif.

Perintah ini kemudian dilanjutkan dengan memuat dataset CSV dari direktori kerja menggunakan `pd.read_csv()` dan menyaring kolom `"processed_text"`. Baris yang memiliki nilai *missing* (*NaN*) dihapus via `dropna()`, teks dipastikan bertipe string serta dibersihkan dari spasi liar di awal/akhir melalui `astype(str).str.strip()`, lalu baris yang tidak kosong disaring dan urutan indeksnya di-reset dengan `reset_index(drop=True)`.

Perintah ini diakhiri dengan proses ekstraksi fitur dan pemodelan bahasa. Objek `CountVectorizer()` dibentuk untuk mengubah teks menjadi matriks frekuensi kata `(bow_matrix)`, yang kemudian ditransformasikan lebih lanjut menjadi matriks pembobotan TF-IDF menggunakan `TfidfTransformer()` `(tfidf_matrix)`. Sebagai penutup, program mencetak jumlah total dokumen yang terproses (119.613 dokumen) serta total ukuran kosa kata unik yang berhasil diekstrak (32.654 kata).

```python
# Nomor 1: Pilih 3 dokumen dari dataset. Hitung cosine similarity di antara keduanya menggunakan TF-IDF. Mana yang paling mirip?
from itertools import combinations

doc_indices = [0, 1, 2]  # Ganti indeks jika ingin memilih dokumen lain

selected_tfidf = tfidf_matrix[doc_indices]
similarity_matrix = cosine_similarity(selected_tfidf)

display(pd.DataFrame(
    similarity_matrix,
    index=[f"Dokumen {i}" for i in doc_indices],
    columns=[f"Dokumen {i}" for i in doc_indices],
))

pairs = [
    (similarity_matrix[i, j], doc_indices[i], doc_indices[j])
    for i, j in combinations(range(len(doc_indices)), 2)
]
score, doc_a, doc_b = max(pairs)

print(f"Pasangan paling mirip: dokumen {doc_a} dan {doc_b}")
print(f"Cosine similarity: {score:.4f}")
print("\nTeks dokumen:")
print(f"{doc_a}: {data.loc[doc_a, 'content']}")
print(f"{doc_b}: {data.loc[doc_b, 'content']}")
for nomor, idx in enumerate(doc_indices, start=1):
    print(f"Dokumen {nomor} (indeks {idx}):")
    print(data.loc[idx, "content"])
    print()
```

✓ 0.0s

	Dokumen 0	Dokumen 1	Dokumen 2
Dokumen 0	1.0	0.0	0.0
Dokumen 1	0.0	1.0	0.0
Dokumen 2	0.0	0.0	1.0

```
Pasangan paling mirip: dokumen 1 dan 2
Cosine similarity: 0.0000

Teks dokumen:
1: Bagus dan mudah digunakan
2: Good
Dokumen 1 (indeks 0):
Mantap

Dokumen 2 (indeks 1):
Bagus dan mudah digunakan

Dokumen 3 (indeks 2):
Good
```

Dari hasil, "Bagus dan mudah digunakan" dan "Good" memiliki skor 0,0000. Artinya,

TF-IDF tidak menemukan kata yang sama di antara keduanya.

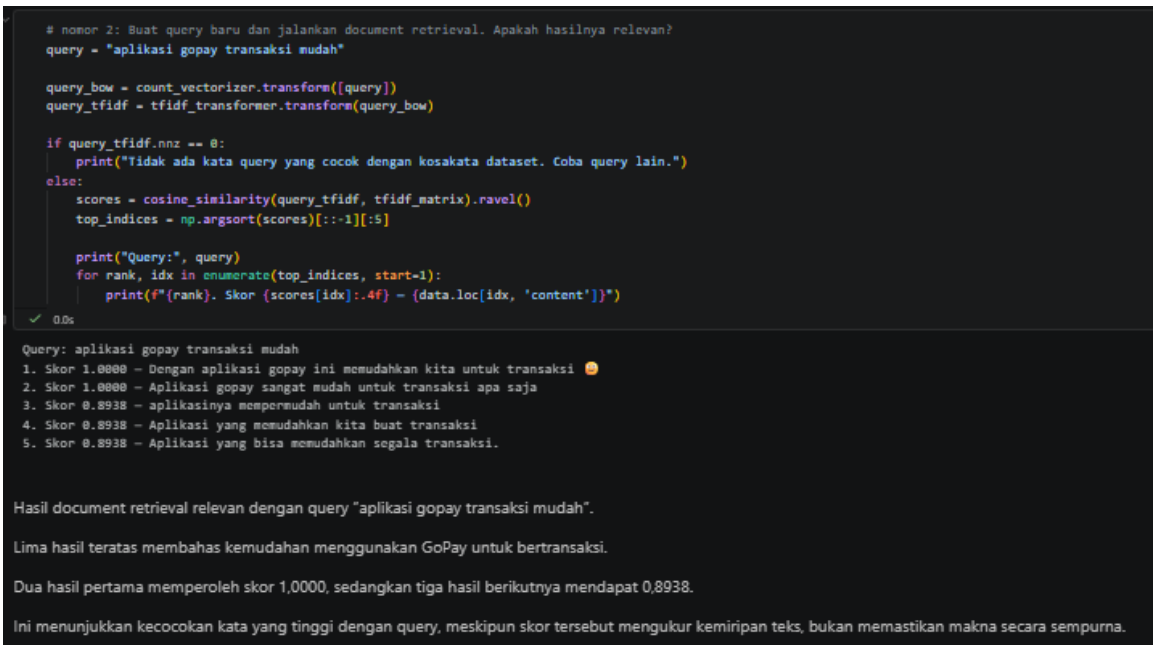
Pada matriks sebelumnya semua skor di luar diagonal juga 0.0,

jadi ketiga pasangan sama rendah kemiripannya—belum ada satu pasangan yang paling mirip secara unik.

Perintah ini diawali dengan mengimpor modul `combinations` dari `itertools` serta menentukan indeks 3 dokumen yang akan diuji melalui `doc_indices = [0, 1, 2]`. Vektor TF-IDF untuk ketiga dokumen tersebut diekstrak dari tfidf_matrix, lalu tingkat kemiripan antar-dokumen dihitung menggunakan `cosine_similarity()`. Hasil perhitungan kemudian ditampilkan dalam bentuk DataFrame pandas yang rapi dengan label `Dokumen 0`, `Dokumen 1`, dan `Dokumen 2`.

Perintah ini kemudian dilanjutkan dengan mencari pasangan dokumen yang paling mirip. Seluruh kombinasi pasangan unik dibuat menggunakan `combinations(range(len(doc_indices)), 2)`, lalu skor maksimum beserta pasangan indeks dokumennya diambil via fungsi `max(pairs)`. Kode kemudian mencetak informasi pasangan dokumen paling mirip beserta nilai skor kemiripannya, dilanjutkan dengan mencetak teks asli dari pasangan dokumen tersebut dari kolom `'content'`.

Perintah ini diakhiri dengan menjalankan perulangan `for` untuk mencetak teks lengkap dari ketiga dokumen sampel. Berdasarkan output yang dihasilkan, nilai *cosine similarity* di luar diagonal bernilai `0.0`, yang menunjukkan bahwa ketiga dokumen (Dokumen 0: "Mantap", Dokumen 1: "Bagus dan mudah digunakan", Dokumen 2: "Good") tidak memiliki kosakata yang saling tumpang tindih sehingga tingkat kemiripannya sama-sama rendah (0,0000).

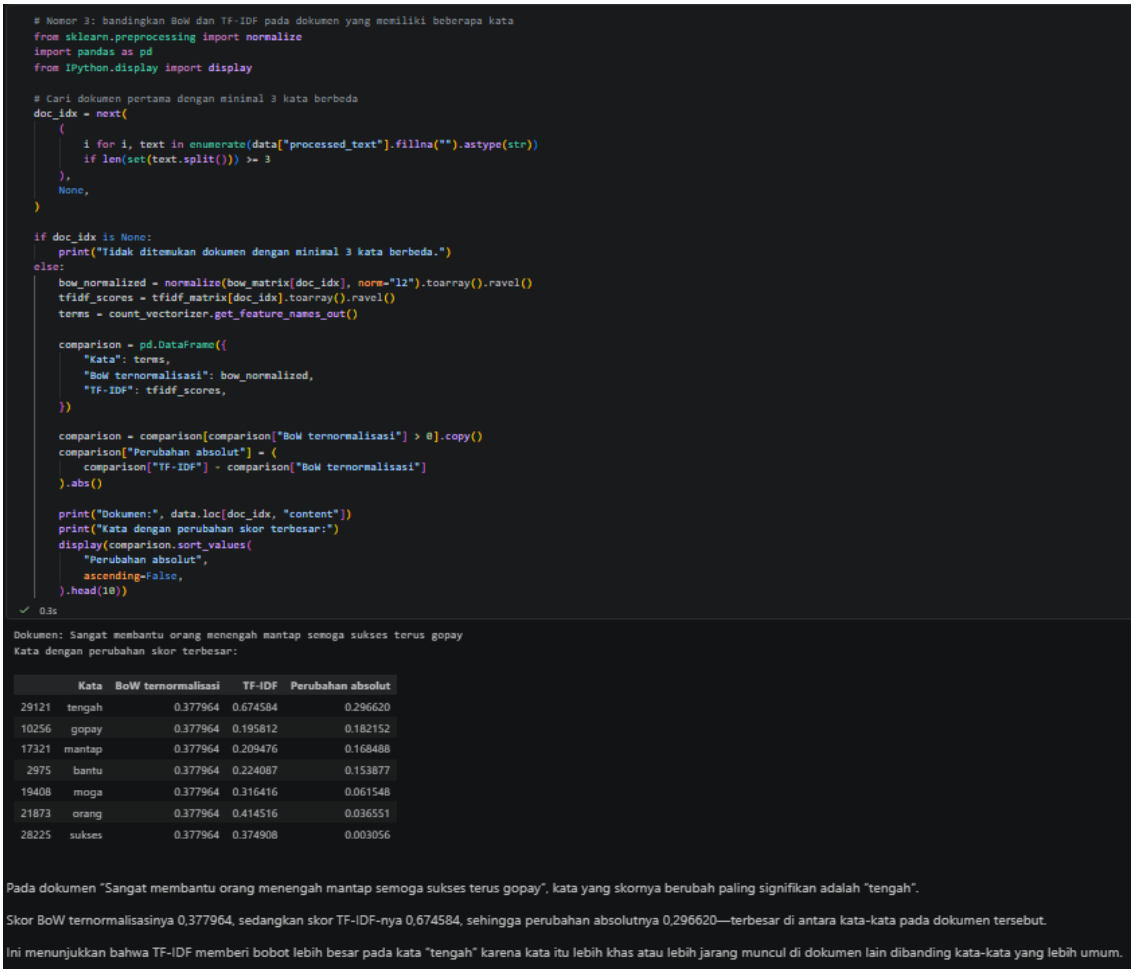


Perintah ini diawali dengan mendefinisikan teks pencarian baru `query = "aplikasi gopay transaksi mudah"`. Teks *query* tersebut kemudian ditransformasikan ke bentuk vektor Bag of Words menggunakan `count_vectorizer.transform()`. Selanjutnya, vektor BoW tersebut diubah menjadi representasi pembobotan TF-IDF melalui perintah `query_tfidf = tfidf_transformer.transform(query_bow)`.

Perintah ini kemudian dilanjutkan dengan memeriksa keberadaan kata *query* dalam kosakata dataset menggunakan pengecekan `if query_tfidf.nnz == 0:`. Jika ada kata yang cocok, blok `else:` dijalankan untuk menghitung tingkat kemiripan *query* terhadap seluruh dokumen dalam `tfidf_matrix` menggunakan `cosine_similarity()`. Indeks 5 dokumen teratas dengan skor kemiripan tertinggi diekstrak secara menurun melalui `top_indices = np.argsort(scores)[::-1][:5]`. Hasil pencarian kemudian dicetak

menggunakan perulangan for untuk menampilkan urutan peringkat, skor kemiripan, serta isi teks dokumen asli dari kolom `'content'`.

Perintah ini diakhiri dengan mengevaluasi relevansi dokumen hasil pencarian. Output menunjukkan bahwa kelima hasil teratas sangat relevan dengan topik kemudahan transaksi GoPay, di mana dua dokumen pertama memperoleh skor kemiripan sempurna sebesar 1.0000 dan tiga dokumen berikutnya memperoleh skor 0.8938. Hasil ini membuktikan kecocokan kata kunci (*term matching*) yang tinggi pada sistem *document retrieval* berbasis TF-IDF.



```
# Nomor 3: bandingkan BoW dan TF-IDF pada dokumen yang memiliki beberapa kata
from sklearn.preprocessing import normalize
import pandas as pd
from IPython.display import display

# Cari dokumen pertama dengan minimal 3 kata berbeda
doc_idx = next(
    (
        i for i, text in enumerate(data["processed_text"].fillna("").astype(str))
        if len(set(text.split())) >= 3
    ),
    None,
)

if doc_idx is None:
    print("Tidak ditemukan dokumen dengan minimal 3 kata berbeda.")
else:
    bow_normalized = normalize(bow_matrix[doc_idx], norm="l2").toarray().ravel()
    tfidf_scores = tfidf_matrix[doc_idx].toarray().ravel()
    terms = count_vectorizer.get_feature_names_out()

    comparison = pd.DataFrame({
        "Kata": terms,
        "BoW ternormalisasi": bow_normalized,
        "TF-IDF": tfidf_scores,
    })

    comparison = comparison[comparison["BoW ternormalisasi"] > 0].copy()
    comparison["Perubahan absolut"] = (
        comparison["TF-IDF"] - comparison["BoW ternormalisasi"]
    ).abs()

    print("Dokumen:", data.loc[doc_idx, "content"])
    print("Kata dengan perubahan skor terbesar:")
    display(comparison.sort_values(
        "Perubahan absolut",
        ascending=False,
    ).head(10))
```

✓ 0.3s

```
Dokumen: Sangat membantu orang menengah mantap semoga sukses terus gopay
Kata dengan perubahan skor terbesar:
```

	Kata	BoW ternormalisasi	TF-IDF	Perubahan absolut
29121	tengah	0.377964	0.674584	0.296620
10256	gopay	0.377964	0.195812	0.182152
17321	mantap	0.377964	0.209476	0.168488
2975	bantu	0.377964	0.224087	0.153877
19408	moga	0.377964	0.316416	0.061548
21873	orang	0.377964	0.414516	0.036551
28225	sukses	0.377964	0.374908	0.003056

Pada dokumen "Sangat membantu orang menengah mantap semoga sukses terus gopay", kata yang skornya berubah paling signifikan adalah "tengah".

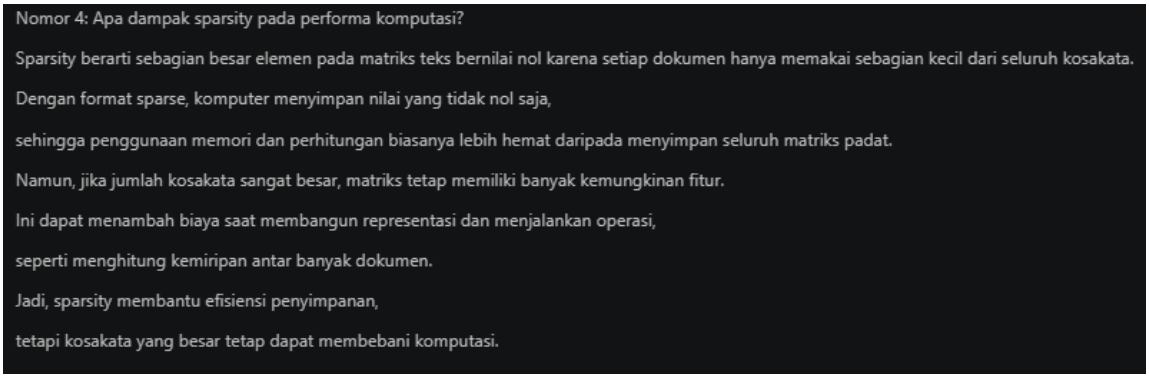
Skor BoW ternormalisasinya 0,377964, sedangkan skor TF-IDF-nya 0,674584, sehingga perubahan absolutnya 0,296620—terbesar di antara kata-kata pada dokumen tersebut.

Ini menunjukkan bahwa TF-IDF memberi bobot lebih besar pada kata "tengah" karena kata itu lebih khas atau lebih jarang muncul di dokumen lain dibanding kata-kata yang lebih umum.

Perintah ini diawali dengan mengimpor modul yang dibutuhkan (`normalize`, `pandas`, dan `display`). Selanjutnya, kode mencari indeks dokumen pertama yang memiliki minimal 3 kata unik menggunakan fungsi `next()`. Pencarian dilakukan dengan memecah teks berdasarkan spasi `(split())` lalu menghitung jumlah elemen uniknya dengan `set()` pada kolom `processed_text`.

Perintah ini kemudian dilanjutkan dengan mengekstrak dan menormalisasi vektor kata. Nilai Bag of Words pada dokumen yang terpilih dinormalisasi menggunakan L2-norm via `normalize(..., norm="l2")`, sedangkan skor TF-IDF-nya diambil langsung dari `tfidf_matrix`. Kedua representasi ini disatukan ke dalam DataFrame pandas bersama daftar kata (`terms`), lalu disaring hanya untuk kata-kata yang benar-benar muncul dalam dokumen (`BoW ternormalisasi > 0`). Sebuah kolom baru bernama `Perubahan absolut` dibuat untuk mengukur selisih mutlak antara skor TF-IDF dan BoW ternormalisasi.

Perintah ini diakhiri dengan mencetak teks asli dokumen dari kolom `content` dan menampilkan tabel perbandingan 10 kata dengan perubahan skor terbesar yang diurutkan secara menurun (`sort_values`). Berdasarkan output, kata "`tengah`" mengalami perubahan paling signifikan dengan selisih mutlak 0,296620 (skor BoW 0,377964 naik menjadi 0,674584 pada TF-IDF). Hal ini membuktikan bahwa metode TF-IDF memberikan bobot lebih tinggi pada kata yang unik/jarang muncul di dokumen lain dibandingkan kata umum seperti "`gopay`".



F. Pertanyaan Diskusi

1. Mengapa BoW tidak mempertimbangkan urutan kata, dan apakah hal tersebut menjadi masalah?

   Jawaban: BoW hanya menghitung jumlah kemunculan kata tanpa memperhatikan urutan kata dalam kalimat. Hal ini bisa menjadi masalah karena makna kalimat dapat berbeda meskipun menggunakan kata yang sama, contohnya "saya suka GoPay" dan "GoPay tidak saya suka".

2. Kata apa yang mendapat skor TF-IDF rendah meskipun sering muncul? Mengapa?

   Jawaban: Kata seperti "dan", "yang", "di", dan "dengan" biasanya mendapat skor TF-IDF rendah karena sering muncul di banyak dokumen. Kata-kata tersebut kurang membantu membedakan satu dokumen dengan dokumen lainnya.

3. Apakah cosine similarity cocok untuk semua jenis teks?

   Jawaban: Tidak selalu. Cosine similarity cocok untuk membandingkan kemiripan arah vektor teks, tetapi hasilnya bergantung pada representasi teks yang digunakan. Metode ini juga belum tentu memahami makna kalimat atau konteks secara mendalam.

4. Mengapa matriks BoW/TF-IDF sangat sparse? Apa dampaknya?

   Jawaban: Matriks BoW/TF-IDF sangat sparse karena setiap dokumen hanya menggunakan sebagian kecil kata dari seluruh vocabulary. Akibatnya, sebagian besar nilai matriks adalah nol sehingga ukuran data bisa besar dan membutuhkan lebih banyak sumber daya untuk diproses.

5. Apa keterbatasan TF-IDF yang tidak dapat diselesaikan tanpa word embedding

   Jawaban: TF-IDF tidak memahami hubungan makna antar kata. Kata yang memiliki arti mirip, seperti "bagus" dan "baik", dianggap berbeda. Word embedding membantu mengatasi keterbatasan tersebut dengan merepresentasikan kata sebagai vektor yang dapat menunjukkan kemiripan makna.

G. Kesimpulan

Kesimpulan yang dapat diperoleh dari praktikum ini adalah:

1. Teks perlu diubah menjadi bentuk numerik agar dapat diproses oleh komputer.
2. Bag of Words (BoW) digunakan untuk menghitung frekuensi kemunculan kata, sedangkan TF-IDF digunakan untuk menentukan tingkat kepentingan kata dalam dokumen.
3. Cosine similarity digunakan untuk mengukur kemiripan antar dokumen, sedangkan document retrieval digunakan untuk mencari dokumen yang relevan dengan kata kunci tertentu.
4. BoW dan TF-IDF memiliki keterbatasan karena tidak memahami urutan kata dan makna kata secara mendalam.
5. Word embedding dapat digunakan untuk merepresentasikan hubungan makna antar kata dengan lebih baik.